Joint-Embedding Predictive Architecture

Zachary McCaw

2026-11-05

VICReg

VICReg [1] learns representations by generating two independently augmented views x and x' of the same input, encoding them as $y = f_\theta(x)$ and $y' = f_\theta(x')$, and then passing these representations through a disposable projection head h_ϕ to obtain $z = h_\phi(y)$ and $z' = h_\phi(y')$. In VICReg this head is specifically an *expander*: rather than projecting to a lower-dimensional space, it maps the encoder representation to a substantially higher-dimensional embedding space (e.g., from 2048 to 8192 dimensions in the standard ImageNet setup). The VICReg loss is applied to the expanded embeddings rather than directly to the representations and combines invariance, variance, and covariance terms. For batches $Z = [z_1, \dots, z_n]$ and $Z' = [z'_1, \dots, z'_n]$ of d-dimensional embeddings, these are

$$s(Z, Z') = \frac{1}{n} \sum_{i=1}^{n} \|z_i - z'_i\|_2^2,$$

$$v(Z) = \frac{1}{d} \sum_{j=1}^{d} \max \left\{ 0, \gamma - \sqrt{\mathrm{Var}(z^j) + \epsilon} \right\},$$

$$C(Z) = \frac{1}{n-1} \sum_{i=1}^{n} (z_i - \bar{z})(z_i - \bar{z})^\top, \qquad c(Z) = \frac{1}{d} \sum_{j \neq k} [C(Z)]_{jk}^2,$$

$$\mathcal{L}_{\text{VICReg}}(Z, Z') = \lambda s(Z, Z') + \mu\{v(Z) + v(Z')\} + \nu\{c(Z) + c(Z')\}.$$

The invariance term makes the embeddings of two views of the same input similar; the variance term requires each embedding coordinate to maintain sufficient variation across samples, thereby preventing collapse; and the covariance term discourages redundancy by driving pairwise covariances between embedding coordinates toward zero. The paper uses $\gamma = 1$ and, for ImageNet, $\lambda = \mu = 25$ and $\nu = 1$.

After pretraining, the expansion head h_ϕ is discarded and the encoder representation y is retained for downstream use. Consequently, the variance and covariance constraints are not imposed directly on the representation of ultimate interest. This separation is intentional: the expansion head provides an auxiliary space in which strong invariance and redundancy-reduction constraints can be enforced without requiring y itself to satisfy those constraints exactly. Nevertheless, gradients from the VICReg objective propagate through h_ϕ into the encoder, and the paper empirically finds that covariance regularization applied at the expanded embedding level reduces correlation among dimensions of the encoder representation as well. Thus, in practice, VICReg is naturally implemented

by attaching a disposable, preferably relatively wide, nonlinear expansion head to the representation being learned, applying the VICReg objective to the expanded embeddings during training, and retaining the pre-expansion representation for downstream tasks.

VICRegL

VICRegL [2] extends VICReg by applying the same variance–invariance–covariance objective at both global and local levels of a convolutional encoder. Given two augmented views of the same image, the encoder produces spatial feature maps $Y, Y' \in \mathbb{R}^{C\times H\times W}$. For the global criterion, these feature maps are mean-pooled to vectors, passed through the standard VICReg expander, and regularized with the usual VICReg loss. For the local criterion, the unpooled feature maps are passed through a smaller local projector that preserves their spatial arrangement, after which two separate local VICReg losses are constructed using different matching rules:

1. **Location-based matching:** each local feature is mapped back to the approximate position in the original seed image from which it arose, using the known coordinates of the crop and whether the crop was horizontally flipped. It is matched to the feature in the other view whose corresponding position in the original image is closest. Among the resulting matched pairs, only the top-γ pairs are retained, and VICReg is applied to these pairs to define a location-based local loss L_s.

2. **Feature-based matching:** each local embedding is matched to its nearest neighbor in the other view according to Euclidean distance in embedding space, allowing semantically similar regions to be matched even when they occur at different spatial locations. Again, only the top-γ matched pairs are retained, and VICReg is applied to them to define a separate feature-based local loss L_d.

Because nearest-neighbor matching is directional, both local losses are symmetrized by computing matches in both directions, i.e. from the first view to the second and from the second view to the first. In the standard two-view setup, the paper uses $\gamma = 20$; with multi-crop, it uses $\gamma_1 = 20$ for large views and $\gamma_2 = 4$ for small views. The final objective is a weighted combination of the global VICReg loss and the two symmetrized local losses, with α controlling the emphasis on global versus local representation learning; the main experiments use $\alpha = 0.9$ or $\alpha = 0.75$, with $\alpha = 0.75$ serving as a representative setting that places somewhat greater emphasis on local features.

LeCun Perspective

LeCun's perspective [3] proposes a general architecture for autonomous intelligence centered on learned world models, latent-state prediction, and planning. For the purposes of world and dynamics modeling, the most important ideas are the following.

1. **World models should learn action-conditioned latent dynamics.** In the control setting, the relevant transition object is not merely $p(s_{t+1} \mid s_t)$, but rather

$p(s_{t+1} \mid s_t, a_t)$. If actions are omitted, then the learned transition law is implicitly averaged over the behavior policy that generated the data:

$$p(s_{t+1} \mid s_t) = \int p(s_{t+1} \mid s_t, a_t) p(a_t \mid s_t)\, da_t.$$

For planning, LeCun instead envisions a predictor of the form

$$s_{t+1} = \mathrm{Pred}(s_t, a_t, z_t),$$

where z_t represents structured residual uncertainty in the next state that is not determined by (s_t, a_t).

2. **Prediction should occur in representation space rather than directly in observation space.** A JEPA encodes an observed/context variable x and a target/future variable y as $s_x = g_x(x)$ and $s_y = g_y(y)$, then predicts the target representation using $\widetilde{s}_y = \mathrm{Pred}(s_x, z)$. The corresponding energy is

$$E_w(x, y, z) = D\{s_y, \mathrm{Pred}(s_x, z)\},$$

with compatibility defined by

$$F_w(x, y) = \min_{z \in \mathcal{Z}} E_w(x, y, z).$$

This allows the target encoder to discard unpredictable nuisance detail rather than forcing the world model to reproduce every low-level aspect of y.

3. **When the future is not uniquely determined, model compatibility among plausible outcomes rather than force a single point prediction.** The latent variable z parameterizes alternative plausible relationships between x and y, while invariances in the target encoder can eliminate variation that need not be represented at all. Conceptually:

   - Predictable structure should be retained in s_y.
   - Relevant but unpredictable variation should be represented through z.
   - Irrelevant variation should be discarded by the encoder.

   Thus z can be interpreted as a structured residual component that indexes ambiguity in $y \mid x$.

4. **Predictability creates an anti-collapse problem.** If the only objective were to make s_y easy to predict from s_x, the trivial solution would be to make both representations constant. JEPA therefore requires opposing constraints that keep s_x and s_y informative about their inputs. LeCun proposes four simultaneous criteria:

   - Maximize the information content of s_x about x.
   - Maximize the information content of s_y about y.
   - Make s_y easy to predict from s_x.

- Minimize the information content of z.

VICReg is presented as one practical non-contrastive realization of the first two criteria: variance regularization prevents individual coordinates from becoming constant, while covariance regularization discourages redundancy among coordinates. The latent z must also have limited capacity; otherwise the predictor could ignore s_x and encode the full target through z.

5. **World models should operate hierarchically across both abstraction levels and time scales.** LeCun proposes H-JEPA, in which lower levels retain more detailed state information and support short-horizon prediction, while higher levels learn progressively more abstract representations suitable for longer-horizon prediction. For example, a hierarchy may take the form

$$s_t^{(1)} = \mathrm{Enc}_1(x_t), \quad s_t^{(2)} = \mathrm{Enc}_2\left(s_t^{(1)}\right).$$

Higher levels then predict over increasingly long horizons. The key idea is that long-horizon prediction requires discarding fast-varying detail while retaining slower, more stable structure.

6. **The purpose of the world model is planning.** Given a differentiable predictor and cost function, the actor can simulate candidate action sequences, evaluate the resulting predicted trajectories, and optimize actions using gradients of predicted future cost with respect to the proposed actions. A representative objective is

$$J(a_{0:T}) = \sum_{t=1}^{T} C(s_t).$$

At higher levels of the hierarchy, LeCun's "abstract actions" are better interpreted as latent subgoals or desired future-state conditions that lower levels must realize. In this way, the architecture combines latent-state prediction, hierarchical abstraction, and model-predictive control into a common framework for reasoning and planning.

I-JEPA

I-JEPA [4] instantiates the Joint-Embedding Predictive Architecture for images by predicting masked target representations from a visible context region of the same image. An image is divided into a common grid of non-overlapping patches. The full set of image patches is passed simultaneously through the target encoder $f_{\bar{\theta}}$, which is a Vision Transformer (ViT), and therefore produces context-dependent patch representations through self-attention across the full patch sequence. Target blocks are then selected from the *output* of the target encoder. Independently, a large context block is sampled from the same patch grid; if any portion of the sampled context overlaps a target block, those overlapping patches are removed from the context mask before encoding, while the target block itself is left unchanged. The remaining visible context patches are passed

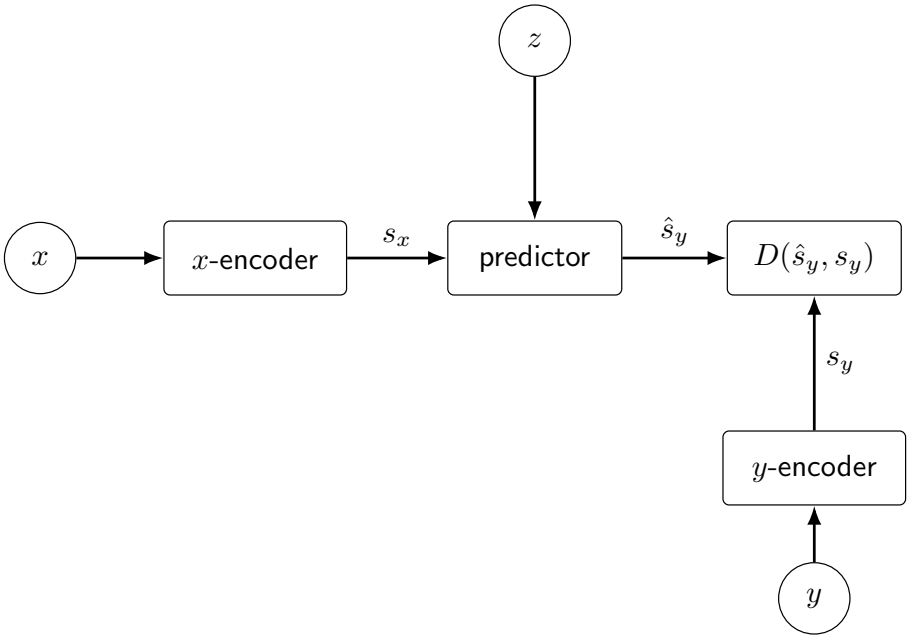


Figure 1: Schematic of a Joint-Embedding Predictive Architecture (JEPA). The context input x is encoded by the x-encoder to produce a representation s_x. The predictor uses s_x together with an auxiliary variable z, which specifies the transformation, corruption, location, or other conditioning information needed to identify the target, to produce a prediction $\hat{s}_y$. Separately, the target input y is encoded by the y-encoder to produce the target representation s_y. Training minimizes a discrepancy $D(\hat{s}_y, s_y)$ in representation space rather than reconstructing y directly in observation space.

simultaneously through the context encoder f_θ, also a ViT. For target block i, let B_i denote its patch indices. The predictor g_ϕ takes the full set of encoded context tokens together with one positional mask token for each patch in B_i. Because the predictor is a ViT, these tokens are processed jointly, and its outputs at the mask-token positions provide predictions $\hat{s}_{y_j}$ for all $j \in B_i$. The predictor is applied separately to each of the M target blocks, and each prediction is compared with the corresponding target-encoder representation s_{y_j}. The loss is computed only over the target patches, not over the visible context patches:

$$\mathcal{L} = \frac{1}{M} \sum_{i=1}^{M} \sum_{j \in B_i} \left\| \hat{s}_{y_j} - s_{y_j} \right\|_2^2 .$$

The context encoder and predictor are updated by gradient descent, whereas the target encoder is updated as an exponential moving average of the context encoder, providing the architectural asymmetry used to avoid collapse. A key image-specific ingredient is the multi-block masking strategy: each target is a contiguous rectangular block covering roughly 15%–20% of the image, while the context block initially covers roughly 85%–100% of the image before any target-overlapping patches are removed; positional tokens specify the locations of the target patches. The use of 2D patch grids, rectangular masks, and image-scale heuristics is specific to vision. The broader ideas of predicting representations rather than raw observations, using separate context and target encoders, conditioning

the predictor on target identity or location, and stabilizing training with a slowly moving target encoder generalize to other modalities. Importantly, I-JEPA is reconstructive in the sense that it predicts representations of masked regions from other regions of the *same static image*; it does not model temporal dynamics or forecast future world states.

V-JEPA

V-JEPA [5] extends the Joint-Embedding Predictive Architecture to video by predicting masked spatiotemporal representations rather than pixels. A video clip of shape $T \times H \times W$ is partitioned into non-overlapping tubelets of size $2 \times 16 \times 16$, so each token spans two consecutive frames and a 16×16 spatial region. For the standard $16 \times 224 \times 224$ input, this yields 1568 spatiotemporal tokens. The context encoder E_θ processes only the visible tokens, while the target encoder $E_{\bar{\theta}}$ processes the full clip and produces contextualized target representations. The predictor P_ϕ receives the context representations together with learnable mask tokens augmented by 3D positional information specifying the masked locations, and predicts the corresponding target representations. Training minimizes an L_1 loss over masked tokens,

$$\mathcal{L} = \frac{1}{|\mathcal{M}|} \sum_{k \in \mathcal{M}} \left\| \hat{s}_k - s_k \right\|_1 .$$

V-JEPA replaces I-JEPA's L_2 regression loss with L_1. The authors report that L_1 was more stable in practice; a plausible interpretation is that its bounded per-residual gradient makes optimization less sensitive to large latent prediction errors, although the paper does not characterize the source of instability in detail. The target branch is stop-gradient and its parameters are updated as an exponential moving average of the context encoder to prevent representation collapse.

V-JEPA uses a 3D multi-block masking strategy in which spatially contiguous blocks are extended across the full temporal duration of the clip. Two masks are sampled per clip: a short-range mask formed from eight smaller blocks and a long-range mask formed from two larger blocks, with an effective masking ratio of approximately 90%. Extending the masks through the full temporal dimension reduces information leakage from neighboring frames. Ablations show that representation quality is improved by high spatial and temporal masking ratios, several smaller blocks rather than a single large block, and using both mask types. The standard objective is non-causal, so V-JEPA is best viewed as spatiotemporal latent completion rather than future-state prediction.

For downstream tasks, the pretrained encoder is frozen and its token sequence is pooled with an *attentive probe*. A learnable query vector q cross-attends to the frozen spatiotemporal representations $(s_1, \ldots, s_L)$,

$$\sum_{i=1}^{L} \frac{\exp\left(q^\top W_k s_i\right)}{\sum_j \exp\left(q^\top W_k s_j\right)} W_v s_i.$$

The resulting pooled representation is passed through a small MLP, LayerNorm, and a linear classifier. The query is introduced only for downstream evaluation and is not tied to masked positions; instead, it learns which spatial and temporal tokens are most informative for the task.

DINO-WM

DINO-WM [6] learns a task-agnostic, action-conditioned world model from offline state–action trajectories without reward supervision or pixel reconstruction. Unlike Dreamer and TD-MPC2, which learn their representations jointly with the world model, DINO-WM uses a frozen, pretrained DINOv2 encoder to extract spatial patch embeddings $z_t = \mathrm{enc}(o_t) \in \mathbb{R}^{N \times E}$. A transformer then predicts future patch embeddings conditioned on previous embeddings and actions:

$$\hat{z}_{t+1} = p_\theta(z_{t-H:t}, a_{t-H:t}), \qquad \mathcal{L}_{\text{pred}} = \|\hat{z}_{t+1} - \mathrm{enc}(o_{t+1})\|_2^2 .$$

The transformer uses frame-level causal attention, allowing each predicted patch to attend to all patches in previous frames while predicting patches within the next frame in parallel. The model is trained with teacher forcing, and an optional decoder can be trained independently for visualization.

At inference time, DINO-WM performs zero-shot goal-conditioned planning. Given a current observation o_0 and goal observation o_g, it optimizes an action sequence using CEM and MPC to minimize the terminal latent distance $\|\hat{z}_T - \mathrm{enc}(o_g)\|_2^2$. No task-specific policy, reward function, or value function is required. Ablations show that maintaining distinct patch embeddings outperforms compressing the entire image into a single embedding, for instance using the DINOv2 `[CLS]` token.

DINO-WM is similar to the subsequent LeWM [7], which likewise learns reward-free, decoder-free dynamics and performs goal-conditioned planning in latent space. The principal distinction is representation learning: DINO-WM relies on a frozen pretrained encoder to prevent collapse, whereas LeWM jointly trains its encoder and dynamics predictor from pixels, using SIGReg to prevent representation collapse.

Navigation World Models

Bar *et al.* [8] introduce the Navigation World Model (NWM), an action-conditioned latent video model for visual navigation. Observations are encoded with a pretrained VAE, $s_t = \mathrm{enc}(x_t)$, and the model learns stochastic latent dynamics $s_{\tau+1} \sim F_\theta(\cdot \mid s_\tau, a_\tau)$. The action includes translation, yaw, and a temporal offset k, so the same model can predict future states over multiple horizons rather than only one-step transitions. When action labels are unavailable, the model can still be trained using temporal offsets alone, which learns the action-marginal transition $p(s_{t+1} \mid s_t)$ and therefore supplies a generic temporal prior rather than identifiable action-conditioned dynamics.

The transition model is a Conditional Diffusion Transformer (CDiT). Cross-time interaction is implemented asymmetrically: tokens from the noisy target frame self-attend and provide queries, while historical latent tokens provide keys and values through cross-attention. This avoids full self-attention over all frames and reduces complexity from quadratic to linear in context length. Action, temporal offset, and diffusion timestep are embedded separately and combined into a conditioning vector, $\xi = \psi_a + \psi_k + \psi_t$, which modulates each transformer block through adaptive layer normalization. Thus, unlike

architectures such as Dreamer that represent actions as tokens, NWM injects action information globally through the scale and shift of normalized token features.

Adaptive layer normalization. Let $U \in \mathbb{R}^{n \times d}$ contain the n target-frame tokens entering a CDiT sublayer. Layer normalization acts independently on each token:

$$\mathrm{LN}(U)_{ij} = \frac{U_{ij} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}, \qquad \mu_i = \frac{1}{d}\sum_{j=1}^{d} U_{ij}, \qquad \sigma_i^2 = \frac{1}{d}\sum_{j=1}^{d}(U_{ij} - \mu_i)^2.$$

A conditioning MLP maps ξ to four pairs of channelwise scale and shift vectors and three residual gates, following the CDiT block diagram:

$$\left(\{\gamma_j, \beta_j\}_{j=1}^{4}, \{\alpha_m\}_{m=1}^{3}\right) = M(\xi), \qquad \mathrm{AdaLN}_j(U; \xi) = \gamma_j \odot \mathrm{LN}(U) + \beta_j.$$

All coefficients depend on ξ, lie in $\mathbb{R}^d$, and are broadcast across tokens. Let C denote the historical context tokens. Omitting the learned attention projections for readability, one CDiT block can be written as

$$\begin{aligned}
U_1 &= U + \alpha_1 \odot \mathrm{SelfAttn}\{\mathrm{AdaLN}_1(U; \xi)\}, \\
\widetilde{C} &= \mathrm{AdaLN}_2(C; \xi), \qquad \widetilde{U}_1 = \mathrm{AdaLN}_3(U_1; \xi), \\
U_2 &= U_1 + \alpha_2 \odot \mathrm{CrossAttn}(\widetilde{U}_1, \widetilde{C}, \widetilde{C}), \\
U_3 &= U_2 + \alpha_3 \odot \mathrm{FFN}\{\mathrm{AdaLN}_4(U_2; \xi)\}.
\end{aligned}$$

The three arguments of CrossAttn provide its queries, keys, and values, respectively. Thus, γ_j and β_j rescale and recenter normalized feature channels before each operation, while α_m controls the strength of the corresponding residual update. Changing the action, temporal offset, or diffusion timestep changes all of these coefficients. The modulation is global because each coefficient is shared across tokens, but attention still produces token-specific responses from their visual content and relationships.

After training, NWM is used as a simulator rather than as a learned policy. Candidate action sequences are generated either by the Cross-Entropy Method or by an external policy such as NoMaD, rolled out through NWM, and ranked according to a task-specific energy function that combines goal similarity with action and state constraints. This cleanly separates the environment dynamics $p(s_{t+1} \mid s_t, a_t)$ from the planning objective, allowing goals and constraints to change at inference without retraining the world model.

PLDM

Sobal *et al.* [9] propose Planning with a Latent Dynamics Model (PLDM), a reward-free approach that learns task-agnostic dynamics from offline state–action trajectories and performs goal-conditioned planning at test time. Unlike methods that learn a policy for reaching arbitrary goals, PLDM uses a learned world model to optimize action sequences directly.

PLDM consists of an encoder h_θ and an ensemble of action-conditioned latent dynamics predictors $f_\theta^{(k)}$. Given an offline trajectory, the encoder produces $z_t = h_\theta(s_t)$, and each predictor generates an autoregressive latent rollout:

$$\widehat{z}_0^{(k)} = h_\theta(s_0), \qquad \widehat{z}_t^{(k)} = f_\theta^{(k)}\left(\widehat{z}_{t-1}^{(k)}, a_{t-1}\right), \qquad k = 1, \ldots, K.$$

The model is trained by minimizing the discrepancy between predicted and observed latent trajectories, together with VICReg-inspired regularization:

$$\mathcal{L}_{\text{JEPA}} = \mathcal{L}_{\text{sim}} + \alpha\mathcal{L}_{\text{var}} + \beta\mathcal{L}_{\text{cov}} + \delta\mathcal{L}_{\text{time-sim}} + \omega\mathcal{L}_{\text{IDM}}.$$

Here $\mathcal{L}_{\text{sim}}$ penalizes latent prediction errors, the variance and covariance terms discourage representation collapse and redundancy, temporal similarity encourages smooth latent dynamics, and the inverse-dynamics loss preserves action-relevant information. At test time, MPC optimizes action sequences using a latent goal-distance objective penalized by ensemble disagreement, discouraging plans that enter regions where the learned dynamics are uncertain.

V-JEPA 2

V-JEPA 2 [10] retains the masked latent-prediction objective of V-JEPA, but replaces absolute 3D sine-cosine positional embeddings with 3D RoPE and introduces a progressive-resolution training schedule. The underlying pretraining objective remains non-causal masked prediction in latent space, so the pretrained model is primarily a spatiotemporal representation learner rather than an action-conditioned dynamics model.

V-JEPA 2-AC introduces action conditioning in a separate post-training stage. The pretrained encoder E is frozen, and a new predictor P_ϕ is trained to model latent dynamics. Writing $z_t = E(x_t)$ for the encoded observation, s_t for auxiliary state information, and a_t for the action, the predictor models

$$\hat{z}_{t+1} = P_\phi\left[\{(z_k, s_k, a_k)\}_{k \le t}\right].$$

The predictor uses block-causal attention and therefore conditions on the preceding history available within the training sequence; in the implementation, training uses 16-frame clips, so later predictions can condition on all earlier latent, state, and action tokens in the clip. Importantly, the masked-video predictor learned during V-JEPA 2 pretraining is not reused; only the encoder is transferred, while the action-conditioned predictor is trained anew.

Training combines teacher forcing with an autoregressive rollout loss. The teacher-forcing objective predicts each next latent representation from the ground-truth history,

$$\mathcal{L}_{\text{TF}}(\phi) = \frac{1}{T}\sum_{t=1}^{T} \left\| P_\phi\left[\{(z_k, s_k, a_k)\}_{k \le t}\right] - z_{t+1} \right\|_1.$$

The rollout loss instead feeds predicted representations back into the model,

$$\mathcal{L}_{\text{rollout}}(\phi) = \left\| P_\phi(a_{1:H}; s_1, z_1) - z_{H+1} \right\|_1.$$

In V-JEPA 2-AC, the rollout horizon is set to $H = 2$, so gradients pass through one recurrent prediction step. The overall objective is $\mathcal{L} = \mathcal{L}_{\text{TF}} + \mathcal{L}_{\text{rollout}}$, combining accurate one-step prediction with explicit training against autoregressive error accumulation.

Planning

For planning, let $z_t = E(x_t)$ denote the current latent visual representation, s_t the current auxiliary state, $z_g = E(x_g)$ the latent representation of a goal observation, and $a_{1:H} = (a_1, \ldots, a_H)$ a candidate sequence of H future actions, with each $a_h \in \mathbb{R}^d$. The action-conditioned predictor rolls the current state forward under $a_{1:H}$, and the trajectory is scored by the distance between its predicted terminal representation and the goal,

$$J(a_{1:H}) = \left\| P_\phi(a_{1:H}; s_t, z_t) - z_g \right\|_1 .$$

The Cross-Entropy Method (CEM) approximately minimizes J by maintaining a factorized Gaussian proposal over the action coordinates,

$$q_r(a_{1:H}) = \prod_{h=1}^{H} \prod_{j=1}^{d} \mathcal{N}\left(a_{hj}; \mu_{hj}^{(r)}, \sigma_{hj}^{2,(r)}\right),$$

where r indexes the CEM iteration, h the planning step, and j the action coordinate. At iteration r, N candidate trajectories $a_{1:H}^{(1)}, \ldots, a_{1:H}^{(N)}$ are sampled from q_r and evaluated using J. If $\mathcal{E}_r$ denotes the indices of the K lowest-cost trajectories, the proposal parameters are updated using their empirical moments,

$$\mu_{hj}^{(r+1)} = \frac{1}{K} \sum_{n \in \mathcal{E}_r} a_{hj}^{(n)},$$
$$\sigma_{hj}^{2,(r+1)} = \frac{1}{K} \sum_{n \in \mathcal{E}_r} \left[a_{hj}^{(n)} - \mu_{hj}^{(r+1)} \right]^2 .$$

Repeated sampling and refitting concentrates the proposal around low-cost action sequences. Although a multi-step trajectory is optimized, planning is receding-horizon: only the first action is executed, the new state is observed, and the optimization is then repeated from that state. In the reported implementation, the CEM proposal is initialized with zero mean and unit variance for each planning problem rather than warm-started from the proposal obtained at the previous control step.

LeJEPA/SIGReg

LeJEPA [11] develops a JEPA objective from two principles: representations of related views should be predictable from one another, and the embedding distribution should remain non-degenerate. The paper provides theoretical results for linear probes and several nonlinear probe settings showing that isotropic Gaussian embeddings are optimal under the corresponding downstream-risk criteria. This motivates directly regularizing the latent distribution toward an isotropic Gaussian rather than relying on teacher–student networks, stop-gradient, negative samples, or whitening heuristics.

To enforce Gaussianity efficiently in a K-dimensional embedding space, LeJEPA introduces *Sketched Isotropic Gaussian Regularization* (SIGReg). Let $z_n = f_\theta(x_n)$ and let $w \in \mathbb{S}^{K-1}$ denote a unit projection direction. The Cramér–Wold principle states that two multivariate distributions are equal if and only if all of their one-dimensional linear projections are equal in distribution. In particular,

$$w^\top Z \sim N(0,1) \text{ for every } w \in \mathbb{S}^{K-1} \quad \Longleftrightarrow \quad Z \sim N(0, I).$$

SIGReg therefore samples a finite collection of projection directions W and measures how far the projected samples $\{w^\top z_n\}_{n=1}^N$ depart from $N(0,1)$,

$$\text{SIGReg}\left[\{z_n\}_{n=1}^N\right] = \frac{1}{|W|} \sum_{w \in W} T\left[\{w^\top z_n\}_{n=1}^N\right],$$

where T is a univariate goodness-of-fit statistic. LeJEPA uses the Epps–Pulley statistic, which compares the empirical characteristic function $\hat{\phi}(t)$ of the projected samples with the standard Gaussian characteristic function $\phi_0(t) = \exp(-t^2/2)$,

$$T_{\text{EP}} = N \int_{-\infty}^{\infty} \left|\hat{\phi}(t) - \phi_0(t)\right|^2 \omega(t)\, dt.$$

Characteristic-function matching is preferred to finite-moment tests because it identifies the full distribution while retaining bounded gradients and curvature. It is preferred to CDF-based tests because the latter require sorting and order statistics, which introduce non-differentiable operations and synchronization that is poorly suited to efficient multi-GPU training. The empirical characteristic function, by contrast, is an average of complex exponentials and can be aggregated efficiently across devices. The integral is approximated numerically by quadrature, and a fresh random set of projection directions W is sampled for each minibatch, so directional coverage accumulates across SGD updates.

The predictive component is defined in terms of multiple related views of each sample. Let $z_{n,v}$ denote the embedding of sample n under view v, and let a subset of V_g views serve as reference views. Defining

$$\mu_n = \frac{1}{V_g} \sum_{v=1}^{V_g} z_{n,v},$$

the prediction loss directly aligns each of the V views with this reference representation,

$$\mathcal{L}_{\text{pred},n} = \frac{1}{V} \sum_{v=1}^{V} \|\mu_n - z_{n,v}\|_2^2.$$

The general requirement is that the views represent semantically related observations whose relevant content should be aligned. In the image experiments, LeJEPA instantiates this using the DINO-style multi-crop construction, where V_g global views are larger image crops and the remaining views may include smaller local crops. When the views are intended to represent the same underlying information, LeJEPA aligns their embeddings directly without a separate predictor. More generally, the paper

argues that a predictor is warranted when the views differ systematically in information content and the transformation between their representations must itself be modeled.

The complete objective combines cross-view prediction with SIGReg applied separately to the embedding distribution of each view,

$$\mathcal{L}_{\text{LeJEPA}} = \frac{\lambda}{V} \sum_{v=1}^{V} \text{SIGReg}\left[\{z_{n,v}\}_{n=1}^{B}\right] + \frac{1-\lambda}{B} \sum_{n=1}^{B} \mathcal{L}_{\text{pred},n}.$$

Thus λ controls the trade-off between cross-view predictability and isotropic-Gaussian regularization. Because SIGReg is used to guard against collapse, neither stop-gradient nor an EMA teacher is required merely as an anti-collapse mechanism. Empirically, LeJEPA is stable across architectures, batch sizes, view configurations, and SIGReg hyperparameters, and its training loss is strongly correlated with downstream linear-probe performance.

C-JEPA

Causal-JEPA [12] is an object-centric JEPA world model that operates on frozen slot-based representations rather than image patches. Each frame is encoded as a fixed-size set of object latents, and a masked Transformer is trained to reconstruct selected object trajectories within the history window while also predicting future object states. The key idea is to mask entire object histories, retaining only an identity anchor, so that the predictor cannot rely primarily on an object's own recent trajectory; instead, it is encouraged to infer the missing state from other objects and auxiliary variables such as actions and proprioception. At inference, history masking is removed and the model is used for ordinary forward latent prediction and CEM-based model predictive control. A central limitation is the dependence on a reliable object-centric decomposition: the method assumes a fixed slot budget whose slots correspond to coherent entities over time, so slot permutation, identity switching, or scenes containing more relevant objects than available slots can undermine both the masking objective and its interaction-based interpretation.

Temporal straightening

Temporal Straightening for latent planning [13] learns a planning-oriented JEPA world model consisting of a sensory encoder, an action encoder, and a latent dynamics predictor. The sensory encoder maps observations to latent states, $z_t = E^s_\phi(o_t)$, while the action encoder maps controls to latent action embeddings, $E^a_\psi(a_t)$. Given a history of latent states and encoded actions, the predictor estimates the next latent state,

$$\widehat{z}_{t+1} = f_\theta\left(\{z_i\}_{i=t-K+1}^{t}, \{E^a_\psi(a_i)\}_{i=t-K+1}^{t}\right).$$

To make the representation geometry more suitable for planning, the method explicitly straightens latent trajectories. For consecutive latent states, define the velocity vectors

$v_t = z_{t+1} - z_t$ and $v_{t+1} = z_{t+2} - z_{t+1}$, and penalize changes in direction using

$$\mathcal{L}_{\text{curv}} = 1 - \frac{v_t^\top v_{t+1}}{\|v_t\|_2 \|v_{t+1}\|_2}.$$

This can be viewed as a scale-normalized discrete curvature penalty: the second difference $z_{t+2} - 2z_{t+1} + z_t = v_{t+1} - v_t$ measures changes in both speed and direction, whereas the cosine penalty isolates directional change. When consecutive velocities have equal norm c, $\|v_{t+1} - v_t\|_2^2 = 2c^2(1 - \cos\theta_t)$, so the curvature loss is directly proportional to the squared discrete second derivative up to scale. Training combines next-latent prediction with this regularizer,

$$\mathcal{L}_{\text{pred}} = \|\widehat{z}_{t+1} - \text{sg}(z_{t+1})\|_2^2, \qquad \mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{curv}},$$

with stop-gradient on the target branch. This asymmetry blocks gradients through the target representation, but it does not mathematically exclude a constant representation: the paper explicitly notes that collapse remains theoretically possible and reports that it was not observed in the main experiments.

For planning, the paper considers gradient-based optimization and the cross-entropy method (CEM). Given the current latent state z_0, a goal embedding z_g, and a candidate action sequence $a_{0:T-1}$, write the learned history-dependent transition schematically as $\widehat{z}_{t+1} = f_\theta(\widehat{z}_t, a_t)$ and roll it out recursively. Planning then minimizes the terminal latent-space goal cost

$$\mathcal{L}_{\text{plan}}(a_{0:T-1}) = \|\widehat{z}_T - z_g\|_2^2 .$$

Gradient-based planning treats the actions themselves as optimization variables and updates them using

$$a_t \leftarrow a_t - \eta \frac{\partial \mathcal{L}_{\text{plan}}}{\partial a_t}.$$

This requires the learned dynamics model to be differentiable with respect to its action inputs, since the planner must compute quantities such as $\partial \widehat{z}_{t+1}/\partial a_t$ and propagate their effect through the entire rollout. For example, with two transitions, $\widehat{z}_1 = f_\theta(z_0, a_0)$ and $\widehat{z}_2 = f_\theta(\widehat{z}_1, a_1)$, so the gradient of the terminal loss with respect to the first action is obtained by the chain rule,

$$\frac{\partial \mathcal{L}_{\text{plan}}}{\partial a_0} = \frac{\partial \mathcal{L}_{\text{plan}}}{\partial \widehat{z}_2} \frac{\partial \widehat{z}_2}{\partial \widehat{z}_1} \frac{\partial \widehat{z}_1}{\partial a_0}.$$

Thus, gradient-based planning is effectively backpropagation through the learned world-model rollout, with the action sequence rather than the network parameters being optimized. CEM instead treats the dynamics model as a black box and is therefore gradient-free, but requires evaluating many sampled action sequences and is substantially less computationally efficient. The main empirical finding is that temporal straightening makes latent Euclidean distance better reflect dynamical progress toward the goal and improves the conditioning of this action-optimization problem, substantially increasing the success of gradient-based planning and often narrowing the performance gap with CEM.

LeWM

LeWorldModel (LeWM) [7] is a reward-free JEPA world model trained end-to-end from offline trajectories of pixel observations and actions. An encoder maps each observation to a compact latent representation, $z_t = \mathrm{enc}_\theta(o_t)$, and an action-conditioned predictor models the latent dynamics, $\widehat{z}_{t+1} = \mathrm{pred}_\phi(z_{t-H+1:t}, a_{t-H+1:t})$. The default encoder is a ViT-Tiny whose final CLS representation is passed through a projection MLP, while the predictor is a causal Transformer with AdaLN action conditioning. Training uses only a teacher-forced next-latent prediction loss and SIGReg,

$$\mathcal{L}_{\mathrm{LeWM}} = \|\widehat{z}_{t+1} - z_{t+1}\|_2^2 + \lambda\, \mathcal{L}_{\mathrm{SIGReg}}(Z),$$

where SIGReg applies Epps–Pulley normality tests to random one-dimensional projections of the encoded latents, encouraging their joint distribution toward an isotropic Gaussian and thereby discouraging representation collapse. Unlike many prior JEPAs, LeWM uses neither a stop-gradient target, an EMA encoder, nor a pretrained visual representation; gradients propagate through the encoder and predictor jointly.

At inference, candidate action sequences are rolled out autoregressively in latent space and optimized with the cross-entropy method to minimize the terminal distance between the predicted latent state and an encoded goal state. Rather than executing a single optimized sequence open-loop, LeWM uses model predictive control: after executing the current optimized action block, the agent observes the environment again, re-encodes its current state, and replans, allowing subsequent actions to correct for accumulated model error or deviations from the predicted trajectory. A key methodological appeal of LeWM is simplicity: end-to-end latent dynamics learning from pixels requires only a one-step prediction objective and a single anti-collapse regularizer.

VISReg

VISReg [14] is a non-contrastive JEPA regularizer that combines the explicit scale control of VICReg with the full-distribution matching principle of LeJEPA. Its central methodological idea is to decouple *scale* from *shape*. Given centered embeddings $\widehat{Z} \in \mathbb{R}^{N\times D}$, VISReg encourages each latent coordinate to have unit standard deviation through

$$\mathcal{L}_{\mathrm{scale}} = \frac{1}{D}\sum_{j=1}^{D}\left[1 - \sigma_j(\widehat{Z})\right]^2.$$

This explicit scale term is motivated in part by a weakness of SIGReg: as the representation collapses toward the zero vector, the gradient of its Epps–Pulley characteristic-function objective also approaches zero, so its corrective signal becomes weakest precisely near complete collapse. By contrast, the derivative of the VISReg scale penalty with respect to the latent scale remains nonzero as $\sigma_j \to 0$. To prevent the subsequent shape objective from also modifying scale, each centered coordinate is standardized using a detached standard deviation,

$$\widetilde{Z}_j = \frac{\widehat{Z}_j}{\mathrm{sg}(\sigma_j) + \epsilon},$$

where sg denotes stop-gradient.

VISReg regularizes the shape of the standardized embedding distribution using random Cramér–Wold projections and the sliced 2-Wasserstein distance. In general, the one-dimensional Wasserstein distance compares two distributions through their quantile functions; for equally weighted empirical distributions, this reduces to matching their sorted order statistics. VISReg therefore samples random unit directions w_k, projects the standardized embeddings onto each direction, and compares the resulting empirical quantiles with fixed standard-normal quantiles q_N,

$$\mathcal{L}_{\text{shape}} = \frac{1}{K} \sum_{k=1}^{K} \left\| \text{sort}(\widetilde{Z} w_k) - q_N \right\|_2^2 .$$

By the Cramér–Wold principle, matching all one-dimensional projected distributions would identify the full multivariate distribution; the random sketches provide a tractable approximation to this objective. Unlike VICReg's covariance penalty, which controls only second-order dependence, this criterion targets the complete distributional shape of the standardized embeddings. VISReg also includes

$$\mathcal{L}_{\text{center}} = \|\mu\|_2^2,$$

which encourages the batch mean μ of the embeddings to approach the zero vector. The complete regularization term is

$$\mathcal{L}_{\text{Reg}} = \lambda_{\text{scale}} \mathcal{L}_{\text{scale}} + \lambda_{\text{shape}} \mathcal{L}_{\text{shape}} + \lambda_{\text{center}} \mathcal{L}_{\text{center}},$$

with equal component weights used by default, although increasing the relative weight on shape can improve performance on long-tailed or low-rank datasets.

For invariance, VISReg follows LeJEPA and directly aligns each view embedding $z_{n,v}$ with the mean μ_n of the global-view embeddings,

$$\mathcal{L}_{\text{pred},n} = \frac{1}{V} \sum_{v=1}^{V} \|\mu_n - z_{n,v}\|_2^2 .$$

The complete objective is

$$\mathcal{L}_{\text{VISReg}} = (1 - \lambda) \mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{Reg}}.$$

The recommended overall regularization weight depends on dataset scale: $\lambda = 0.9$ is suggested as a starting point for ImageNet-1K, while values around 0.6 are used for smaller datasets. In the image experiments, the views are DINO-style global and local crops, so no separate predictor is required for this symmetric-view setting.

A notable distinction from SIGReg concerns both the discrepancy measure and distributed computation. SIGReg uses the Epps–Pulley characteristic-function criterion, whose empirical characteristic function can be averaged across devices by an all-reduce so that each projected normality test can use the global batch. VISReg instead uses quantile matching, which requires sorting and is not amenable to the same linear all-reduce. It therefore samples different random directions on different GPUs and performs

the Wasserstein calculation locally, so additional GPUs increase directional coverage but do not increase the sample size used to estimate any individual projected distribution. Consequently, VISReg trades SIGReg's efficient global-batch estimation for a scale–shape decomposition with stronger corrective gradients near collapse.

Empirically, VISReg and LeJEPA/SIGReg remain broadly comparable. VISReg shows modest gains on standard downstream evaluations, greater robustness with fewer projection slices, and stronger performance on some challenging datasets when the shape term is upweighted; it also reports improved transfer and out-of-distribution performance.

Fast LeWM

Fast LeWorldModel (Fast-LeWM) [15] replaces LeWM's autoregressive one-step latent rollout with direct action-prefix prediction. Given a current latent z_t and candidate action sequence $a_{t:t+H-1}$, a causal action-prefix encoder produces prefix representations $p_{t,1:H}$, where $p_{t,k}$ summarizes only $(a_t, \ldots, a_{t+k-1})$ together with the current state context. A parallel predictor then estimates all future latents directly from the observed anchor state,

$$\widehat{z}_{t+k} = G_\phi(z_t, p_{t,k}), \qquad k = 1, \ldots, H.$$

Training uses dense supervision over all prefix horizons,

$$\mathcal{L}_{\text{prefix}} = \frac{1}{H} \sum_{k=1}^{H} \|\widehat{z}_{t+k} - z_{t+k}\|_2^2, \qquad \mathcal{L} = \mathcal{L}_{\text{prefix}} + \lambda \, \mathcal{L}_{\text{SIGReg}}(Z),$$

so each prefix token is tied to the latent reached after executing the corresponding partial action sequence. This differs from SG-JEPA's multi-step rollout objective in two ways: Fast-LeWM weights all horizons uniformly rather than using geometric discounting, and each $\widehat{z}_{t+k}$ is predicted directly from z_t and its action prefix rather than recursively from earlier predicted latents. By contrast, SG-JEPA's rollout loss uses geometrically discounted weights and deliberately backpropagates through autoregressive prediction chains so that compounded rollout error shapes the learned representation.

The paper's ablations separate the value of the structured prefix representation from the value of dense prefix supervision. Long-Action LeWM simply enlarges LeWM's action block so that a single transition predicts the terminal latent after a long sequence of primitive actions, without explicitly representing nested prefixes. The ablation labeled "Fast-LeWM without dense loss" retains the causal action-prefix encoder but supervises only the final prediction $\widehat{z}_{t+H}$ rather than every intermediate horizon. Its improvement over Long-Action LeWM therefore isolates the benefit of explicitly encoding the ordered action sequence through accumulated prefixes, while the further gain from full Fast-LeWM shows that dense supervision of the intermediate prefix outcomes is also useful. At planning time, Fast-LeWM retains LeWM's CEM-based latent goal matching, but evaluates each candidate sequence with a single prefix-encoding pass and parallel latent prediction rather than a sequential rollout; an optional self-consistency penalty compares the terminal state predicted directly from the full prefix with a terminal estimate obtained through an intermediate prefix decomposition.

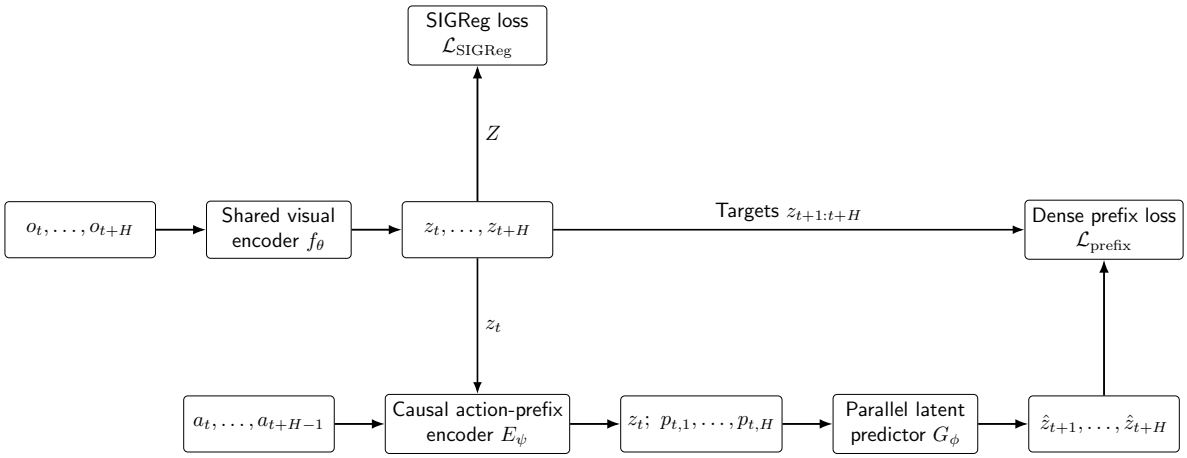


Figure 2: Fast-LeWM training architecture. A shared visual encoder maps the current and future observations $(o_t, \ldots, o_{t+H})$ to latent targets $(z_t, \ldots, z_{t+H})$. The current latent z_t is mapped to a state token and prepended to the candidate action sequence $(a_t, \ldots, a_{t+H-1})$. A causally masked action-prefix encoder produces a token $p_{t,k}$ that summarizes only the first k actions, conditioned on the current state. The parallel latent predictor combines the anchor latent z_t with each prefix token to predict all future latents $(\widehat{z}_{t+1}, \ldots, \widehat{z}_{t+H})$ in one forward pass, without recursively feeding earlier predictions into later ones. Each prediction is matched to its encoded future target through the dense prefix loss, while SIGReg regularizes the encoder outputs to prevent representation collapse. At planning time, the terminal prediction associated with the full action prefix is used to score candidate action sequences against the encoded goal.

SG-JEPA

Semigroup-JEPA (SG-JEPA) [16] learns a latent world model by jointly training a visual encoder and an action-conditioned dynamics predictor using autoregressive multi-step rollouts. Let u_t be the commanded control and g a fixed physical-law parameter, such as gravity. The encoder produces $z_t = e_\phi(o_t)$, while the conditioning encoder receives the augmented input $\widetilde{a}_t = [u_t; g]$ and produces $c_t = q_\psi(\widetilde{a}_t)$. Thus, even when there is no external control, the physical parameter remains available to the dynamics model. Starting from H encoded observations, SG-JEPA recursively feeds its own predictions back into the dynamics model,

$$z_s^{\text{roll}} = \begin{cases} z_s, & s \le t, \\ \widehat{z}_s, & s > t, \end{cases}$$
$$\widehat{z}_{t+k} = f_\theta\left(z^{\text{roll}}_{t+k-H:t+k-1}, c_{t+k-H:t+k-1}\right), \qquad k = 1, \ldots, K.$$

Training minimizes a discounted multi-step latent prediction loss,

$$\mathcal{L}_{\text{roll}} = \sum_{k=1}^{K} w_k \left\| \widehat{z}_{t+k} - z_{t+k} \right\|_2^2, \qquad w_k = \frac{\gamma^{k-1}}{\sum_{j=1}^{K} \gamma^{j-1}},$$

with default rollout horizon $K = 5$ and discount factor $\gamma = 0.95$. Unlike one-step teacher-forced objectives, this exposes the model to errors generated by its own predictions and backpropagates their multi-step consequences through both the dynamics

model and encoder, encouraging representations that remain predictive under repeated rollout.

Because the same trainable encoder generates the prediction targets, a constant latent representation would otherwise provide a trivial solution. SG-JEPA uses SIGReg to guard against representation collapse, applying it only to the encoded latents z_t and yielding

$$\mathcal{L} = \mathcal{L}_{\text{roll}} + \lambda_{\text{sig}} \mathcal{L}_{\text{SIGReg}}.$$

SIGReg regularizes the latent distribution toward a nondegenerate isotropic Gaussian through random one-dimensional projections, while the rollout loss determines which information should be retained for dynamics prediction. The semigroup perspective is that a single learned transition is composed repeatedly under a fixed physical law g, so predictions formed by short compositions should agree with the corresponding longer-interval evolution. Thus, the central methodological distinction of SG-JEPA is that multi-step autoregressive prediction is used not merely to train a better dynamics predictor, but as a representation-learning signal that encourages latent states whose dynamics remain accurate under repeated composition and across physical-law settings.

H-JEPA

Zhang *et al.* [17] introduce H-JEPA, an extension of LeWM that learns a hierarchy of action-conditioned JEPAs operating at progressively longer timescales and in distinct latent spaces. Each level has its own state encoder, action encoder, and causal dynamics predictor. For simplicity, consider a two-level hierarchy in which each macro-step spans two elementary steps; the method generalizes to arbitrarily many levels and configurable temporal strides.

Hierarchical representation learning

At level 1, observations and actions are encoded as $z_t^{(1)} = E^{(1)}(o_t)$ and $a_t^{(1)} = A^{(1)}(a_t)$. Level 2 constructs its representations from the lower level:

$$z_t^{(2)} = E^{(2)}(z_{2t}^{(1)}), \qquad a_t^{(2)} = A^{(2)}(a_{2t}^{(1)}, a_{2t+1}^{(1)}).$$

Thus, the higher-level state encoder transforms every second lower-level state, while its action encoder aggregates two elementary actions into a learned macro-action. The level-2 dynamics predictor consequently learns transitions spanning twice the duration of level-1 transitions.
Each level is trained with teacher-forced latent prediction and SIGReg:

$$\mathcal{L} = \sum_{\ell=1}^{2} \left[\mathcal{L}_{\text{pred}}^{(\ell)} + \lambda_\ell \, \text{SIGReg}(Z^{(\ell)}) \right].$$

Training is end-to-end, without pixel reconstruction, EMA targets, or stop-gradient on latent prediction targets. Because the level-2 encoder operates on level-1 representations, its loss backpropagates through the level-1 encoder, but not through the level-1 dynamics predictor. Longer-horizon prediction encourages the higher-level representation to discard features that become unpredictable over longer intervals while preserving predictable, potentially more abstract state information.

Hierarchical planning

Planning proceeds top-down. Given the current state $z_0^{(2)}$ and goal embedding $g^{(2)}$, the level-2 planner optimizes macro-actions by minimizing the terminal distance of its autoregressive latent rollout:

$$a_{0:H_2-1}^{(2),*} = \arg\min_{a_{0:H_2-1}^{(2)}} \left\| \hat{z}_{H_2}^{(2)} - g^{(2)} \right\|_2^2.$$

The optimized high-level trajectory $(\hat{z}_1^{(2),*}, \ldots, \hat{z}_{H_2}^{(2),*})$ supplies subgoals for level 1. Given candidate elementary actions, the level-1 dynamics predictor generates a rollout, and every second predicted state is mapped into the level-2 latent space for comparison:

$$\tilde{z}_i^{(2)} = E^{(2)}(\hat{z}_{2i}^{(1)}).$$

Two planning regimes are considered:

- **Closed-loop:** Level 1 optimizes two elementary actions to match only the first high-level subgoal, minimizing $\|E^{(2)}(\hat{z}_2^{(1)}) - \hat{z}_1^{(2),*}\|_2^2$. An initial action prefix is executed, the environment is observed, and the hierarchical planning procedure is repeated.
- **Open-loop:** Level 1 optimizes an entire action sequence to match the terminal high-level prediction, optionally also matching intermediate subgoals:

$$\mathcal{C}^{(1)} = \left\| \tilde{z}_{H_2}^{(2)} - \hat{z}_{H_2}^{(2),*} \right\|_2^2 + \beta \sum_{i=1}^{H_2-1} \left\| \tilde{z}_i^{(2)} - \hat{z}_i^{(2),*} \right\|_2^2.$$

At both levels, action sequences are optimized by differentiating the planning cost through the frozen dynamics predictors and updating the actions with AdamW. Upper-level macro-actions are optimized directly as continuous embeddings, initialized from a Gaussian fitted to training macro-actions and clipped to empirical quantile bounds. The elementary actions need only reproduce the predicted high-level states, not the macro-actions themselves; feasibility is not guaranteed. Hierarchical planning decomposes long-horizon optimization into shorter subgoal problems while allowing goal distances to be evaluated in a more abstract latent space.

Inverse-dynamics regularization

In visually diverse environments, a JEPA can exhibit **slow-feature collapse**: the encoder retains static, episode-specific background information while discarding the moving agent. Such representations can satisfy SIGReg through between-episode variation while achieving negligible prediction loss because they remain constant within episodes. To discourage this solution, H-JEPA adds an inverse-dynamics objective that predicts the action connecting consecutive latent states:

$$\mathcal{L}_{\text{IDM}}^{(\ell)} = \frac{1}{T-1} \sum_{t=1}^{T-1} \frac{1}{d_a^{(\ell)}} \left\| I^{(\ell)}(z_t^{(\ell)}, z_{t+1}^{(\ell)}) - \text{sg}(a_t^{(\ell)}) \right\|_2^2.$$

Here $d_a^{(\ell)}$ is the dimension of the level-ℓ action embedding. The total training objective additionally includes $\gamma_\ell \mathcal{L}_{\text{IDM}}^{(\ell)}$ at each level. At level 2, the target is the detached macro-action embedding, so inverse-dynamics gradients update the state encoders but not the action encoder through this target. For DROID, the level-2 macro-action embeddings are also regularized with SIGReg at weight 0.005; no such action term is applied at level 1. Whereas SIGReg maintains a nondegenerate marginal embedding distribution, inverse-dynamics supervision encourages representations to preserve information about action-dependent state transitions.

LeWAM

Hegde *et al.* [18] introduce LeWAM, a reconstruction-free world action model that extends LeWM by jointly learning latent dynamics and a generative action policy. Each observation o_t is encoded into a latent vector $z_t = E_\phi(o_t)$ by a ViT trained from scratch, with SIGReg used to guard against representation collapse by encouraging an isotropic Gaussian latent distribution. A shared bidirectional transformer processes latent-state and action tokens, supporting four prediction modes through input masking:

1. **Forward dynamics:** Predict the next latent state from preceding states and actions, minimizing latent-space MSE.

2. **Backward dynamics:** Predict an earlier latent state from subsequent states and actions, minimizing latent-space MSE.

3. **Inverse dynamics:** Generate the intervening action given states before and after the transition, using rectified flow.

4. **Policy:** Generate the next action chunk and predict the next latent state without observing the latter, combining flow-matching and latent-prediction losses. Preceding actions are masked to prevent copying demonstrated behavior.

All four modes are trained jointly, with gradients propagating through the shared transformer and encoder. Bidirectional attention does not preclude autoregressive world-model rollouts, since future states are masked during forward prediction.

Rectified-flow action generation

Given an observed action chunk $a \in \mathbb{R}^A$ and Gaussian noise $\epsilon \sim \mathcal{N}(0, I_A)$, where A is the action-chunk dimension, training constructs

$$a_\sigma = (1-\sigma)a + \sigma\epsilon, \qquad \sigma \sim \mathcal{U}(0,1),$$

and minimizes the flow-matching loss

$$\mathcal{L}_{\text{flow}} = \mathbb{E}\left[\|v_\theta(a_\sigma, \sigma, c) - (\epsilon - a)\|_2^2\right],$$

where c denotes the available conditioning context. The corrupted action is projected into an action token, augmented with modality, temporal-position, and corruption-level embeddings, schematically $u^a = W_a a_\sigma + e_{\text{action}} + e_{\text{time}} + e(\sigma)$.

At inference, the model starts from $a^{(0)} = \epsilon$ and integrates backward from $\sigma = 1$ to $\sigma = 0$ using eight Euler steps:

$$\sigma_k = 1 - \frac{k}{8}, \qquad a^{(k+1)} = a^{(k)} - \frac{1}{8} v_\theta(a^{(k)}, \sigma_k, c), \qquad k = 0, \ldots, 7.$$

At each step, the updated action estimate and corruption level are supplied to the transformer to recompute the velocity. This defines a deterministic mapping $a = g_\theta(\epsilon; c)$ from Gaussian noise to an action chunk.

Diffusion-steering MPC

Conventional sampling-based MPC, such as CEM or MPPI, optimizes action sequences directly by evaluating their predicted consequences under a learned dynamics model. Such optimization may exploit inaccuracies in the dynamics model by selecting out-of-distribution actions. LeWAM instead optimizes the Gaussian noise inputs $\epsilon_{1:H}$ to the generative policy. For each candidate sequence, actions are decoded at their corresponding imagined contexts, and the forward dynamics model predicts the resulting latent trajectory:

$$\begin{aligned} a_h &= g_\theta(\epsilon_h; \widehat{\mathbf{z}}_{h-1}), \\ \hat{z}_h &= f_\theta(\widehat{\mathbf{z}}_{h-1}, a_h), \\ \widehat{\mathbf{z}}_h &= \mathrm{shift}(\widehat{\mathbf{z}}_{h-1}, \hat{z}_h), \\ \min_{\epsilon_{1:H}} J(\epsilon_{1:H}) &= \|\hat{z}_H - z_g\|_2. \end{aligned}$$

Here $\widehat{\mathbf{z}}_{h-1}$ is the rolling latent context, shift appends the newly predicted state while discarding the oldest element, and z_g is a goal embedding. CEM, MPPI, and CMA-ES retain their usual proposal-update rules, but search in noise space rather than action space, biasing candidate actions toward those generated by the learned policy. To prevent optimization from producing noise vectors with atypically small norms, LeWAM projects each sampled noise vector onto the Gaussian typical-set sphere before decoding:

$$\tilde{\epsilon} = \sqrt{A} \frac{\epsilon}{\|\epsilon\|_2}.$$

The proposal distribution itself remains unconstrained; only sampled candidates are projected. This preserves noise direction while maintaining the typical magnitude of the Gaussian inputs used during policy training.

Optimistic Test-Time World Model Adaptation

Mete *et al.* [19] introduce Optimistic Test-Time Adaptation (OpTTA), a framework for adapting pretrained latent world models during deployment under distribution shift. Unlike conventional test-time adaptation, which minimizes prediction error on observed transitions, OpTTA additionally encourages the latent dynamics model to predict favorable outcomes for candidate action sequences. Inspired by reward-biased maximum likelihood estimation from adaptive control, the approach introduces optimism directly into model estimation without requiring explicit uncertainty estimates.

Optimistic Adaptation Objective

Let E_ϕ denote the observation encoder, f_θ the action-conditioned latent dynamics model, and $z_t = E_\phi(o_t)$ the representation of observation o_t. Given a buffer $\mathcal{B}$ of recently observed transitions (o_t, a_t, o_{t+1}), conventional prediction-based adaptation, as in AdaJEPA, minimizes

$$\mathcal{L}_{\text{pred}} = \frac{1}{|\mathcal{B}|} \sum_{(o_t, a_t, o_{t+1}) \in \mathcal{B}} \| f_\theta(z_t, a_t) - \text{sg}(z_{t+1}) \|^2,$$

where sg denotes stop-gradient. However, because the observed transitions result from actions selected using the current world model, minimizing prediction error on these transitions need not improve predictions for alternative actions that could reach the goal.

OpTTA augments the prediction loss with a task-aware planning objective:

$$\mathcal{L}_{\text{OpTTA}} = \mathcal{L}_{\text{pred}} + \alpha_k \mathcal{L}_{\text{plan}},$$

where

$$\mathcal{L}_{\text{plan}} = \frac{1}{M} \sum_{m=1}^{M} \sum_{j=1}^{H} w_j \| \hat{z}_{t+j}^{(m)} - z_g \|^2.$$

Here M is the number of candidate action sequences, H is the planning horizon, $w_j \geq 0$ weights the cost at step j, $z_g = E_\phi(o_g)$ is the representation of the goal observation, and $\hat{z}_{t+j}^{(m)}$ is the state predicted by recursively applying f_θ under candidate action sequence m. The coefficient $\alpha_k \geq 0$ controls optimism at replanning step k. During adaptation, candidate actions are held fixed while gradients are propagated through the imagined trajectories with respect to the dynamics model parameters. Thus, the planning loss favors models under which candidate actions appear more likely to reach the goal, while the prediction loss anchors adaptation to observed transitions.

Adaptation Procedure

At each replanning step, the agent generates candidate action sequences, executes the first action of the selected plan, observes the resulting transition, and updates the world model using both losses. The planning loss reuses trajectories generated by the planner, requiring no additional environment interactions. It updates only the latent dynamics model, whereas the prediction loss may update both the dynamics model and observation encoder. The action encoder remains fixed.

To balance the two objectives, OpTTA adaptively chooses the optimism weight:

$$\alpha_k = \rho \sqrt{\frac{k}{K_{\max}}} \frac{\| \nabla_{\Omega_f} \mathcal{L}_{\text{pred}} \|_2}{\| \nabla_{\Omega_f} \mathcal{L}_{\text{plan}} \|_2 + \epsilon},$$

where Ω_f denotes the adapted subset of dynamics model parameters, $\rho \geq 0$ controls the relative strength of optimism, $K_{\max}$ is the maximum number of replanning steps, and $\epsilon > 0$ stabilizes the denominator. The coefficient α_k is treated as constant during differentiation. This approximately balances the gradient magnitudes of the prediction and planning terms while gradually increasing the influence of optimism during deployment.

References

[1] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. *arXiv*, 05 2021. https://arxiv.org/abs/2105.04906.

[2] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicregl: Self-supervised learning of local visual features. *arXiv*, 10 2022. https://arxiv.org/abs/2210.01571.

[3] Yann LeCun. A path towards autonomous machine intelligence. *OpenReview*, 06 2022. https://openreview.net/pdf?id=BZ5a1r-kVsf.

[4] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. *arXiv*, 01 2023. https://arxiv.org/abs/2301.08243.

[5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv*, 02 2024. https://arxiv.org/abs/2404.08471.

[6] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv*, 11 2024. https://arxiv.org/abs/2411.04983.

[7] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv*, 03 2026. https://arxiv.org/abs/2603.19312.

[8] Amir Bar, Gaoyue Zhou, Danny Tran, Trevor Darrell, and Yann LeCun. Navigation world models. *arXiv*, 12 2024. https://arxiv.org/abs/2412.03572.

[9] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *arXiv*, 02 2025. https://arxiv.org/abs/2502.14819.

[10] Mahmoud Assran et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv*, 06 2025. https://arxiv.org/abs/2506.09985.

[11] Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics. *arXiv*, 11 2025. https://arxiv.org/abs/2511.08544.

[12] Heejeong Nam, Quentin Le Lidec, Lucas Maes, Yann LeCun, and Randall Balestriero. Causal-jepa: Learning world models through object-level latent masking. *arXiv*, 02 2026. https://arxiv.org/abs/2602.11389.

[13] Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. *arXiv*, 03 2026. https://arxiv.org/abs/2603.12231.

[14] Haiyu Wu, Randall Balestriero, and Morgan Levine. VISReg: Variance-invariance-sketching regularization for JEPA training. *arXiv*, 06 2026. https://arxiv.org/abs/2606.02572.

[15] Yuntian Gao and Xiangyu Xu. Fast leworldmodel. *arXiv*, 06 2026. https://arxiv.org/abs/2606.26217.

[16] Andy Zeyi Liu, Haoran Sun, Lucas Baker, Randall Balestriero, and John Sous. Semigroup-jepa: Latent dynamics consistency for zero-shot physics generalization. *arXiv*, 09 2026. https://arxiv.org/abs/2609.10464.

[17] Wancong Zhang, Basile Terver, Michael Rabbat, Yann LeCun, and Randall Balestriero. H-JEPA: End-to-end learning of hierarchical world models for visual planning. *arXiv preprint arXiv:2610.06805*, 10 2026.

[18] Shashank Hegde, Alexander Popov, Elie Aljalbout, and Nikolai Smolyanskiy. Lewam: A jepa world action model with diffusion-steering-based mpc. *arXiv*, 10 2026.

[19] Akshay Mete, Dileep Kalathil, and P. R. Kumar. Adapt for planning, not just prediction: Optimistic test-time world model adaptation. *arXiv*, 10 2026. https://arxiv.org/abs/2610.35636.